Design and Simulation of a Small Enterprise Network

Applying OSI Layer Functions, IP Subnetting, Routing, and TCP/UDP Protocol Analysis in Cisco Packet Tracer

Computer Networks – Practical / Simulation Report
Tool used: Cisco Packet Tracer 8.2.2

Table of Contents

1. Objective

The goal of this exercise was to build a small enterprise network from scratch inside Cisco Packet Tracer and use it to see, in practice, how the different OSI layers actually cooperate to move data from one host to another. Rather than just drawing a topology and calling it done, we wanted the network to actually do something: route between departments, resolve names through DNS, serve a web page over HTTP, and carry both TCP and UDP traffic so that we could compare how each transport protocol behaves once the network starts to get busy. The report below walks through the theory we relied on, the design decisions, the addressing plan, the full configuration steps for every device, the security touches we added, and finally the results we collected once traffic generation and simulation mode were switched on. Where it's useful, we've also included the actual configuration snippets so the build can be reproduced rather than just described.

2. Background: The OSI Model and Why It Still Matters

Before getting into the build itself, it's worth being clear about why the OSI model was used as the backbone of this whole report rather than jumping straight to configuration commands. The OSI (Open Systems Interconnection) model splits network communication into seven layers, each with a narrow, well-defined job, and each layer only ever talks to the layers directly above and below it. In practice, nobody implements a network that is literally seven independent modules stacked on top of each other — real-world protocol suites like TCP/IP compress several OSI layers together — but the model is still useful because it gives everyone building or troubleshooting a network a shared vocabulary for where a problem actually lives.

The table below shows how the seven OSI layers map onto the four-layer TCP/IP model that the devices in our topology actually run, along with the specific protocols from our build that sit at each layer.

OSI Layer	TCP/IP Layer	Examples From Our Build
Application, Presentation, Session (7, 6, 5)	Application	HTTP, DNS, FTP
Transport (4)	Transport	TCP, UDP
Network (3)	Internet	IP, OSPF, ICMP
Data Link, Physical (2, 1)	Network Access	Ethernet, 802.1Q, serial/HDLC

This mapping matters practically, not just academically. When a PC in the Sales VLAN couldn't reach the file server later in this project (see the troubleshooting log in Section 14), the very first question we asked was 'which layer is this' — because a cabling fault, a VLAN tagging mismatch, a missing route, and a blocked port all look identical from the end user's point of view ("the page won't load") but require completely different fixes. Working top-down through the layers, or bottom-up depending on the symptom, is really the whole value of the model.

3. Network Topology

We modelled a small company with four departments — Administration/HR, Sales & Marketing, an IT/Server room, and a Guest Wi-Fi segment — spread across three physical locations that are joined by two WAN links. The layout is intentionally similar to what a small business with a head office and a branch might actually run.

The core devices used were:

- 3 x Cisco 2911 routers (R1, R2, R3) connected in a partial chain — R1–R2 and R2–R3 — using serial point-to-point links with clock rate set on the DCE end.
- 3 x Cisco 2960 switches, one per site, each carrying multiple VLANs.
- 1 x Wireless Access Point for the Guest VLAN.
- 1 x Server (multi-purpose) at the IT site running DNS, HTTP, and DHCP services.
- A mix of PCs and laptops distributed across the VLANs to generate and receive traffic.

Because the switches carry more than one VLAN, each router uses sub-interfaces on its LAN-facing FastEthernet port (router-on-a-stick), with 802.1Q trunking configured on the switch side. This let us keep the departments logically separated at Layer 2 while still routing between them at Layer 3 through a single physical link, which is a fairly realistic way small offices actually do it rather than giving every VLAN its own physical router port.

4. IP Addressing and Subnetting Scheme

We started from a single private block, 192.168.10.0/24, and used VLSM (variable-length subnet masking) to split it according to how many hosts each segment actually needed, instead of handing every VLAN an equal-sized /24 or /26 and wasting addresses. HR and Sales were sized generously since they hold the bulk of the staff, IT was kept smaller since it mostly just hosts servers and networking gear, the guest network got a small /28 since it is meant to be temporary and low-trust, and the two router-to-router links only need two usable addresses each, so they were given /30s.

VLAN / LAN	Department	Network Address	Usable Hosts	Default Gateway
VLAN 10	Administration / HR	192.168.10.0/26	62	192.168.10.1
VLAN 20	Sales & Marketing	192.168.10.64/26	62	192.168.10.65
VLAN 30	IT / Server Farm	192.168.10.128/27	30	192.168.10.129
VLAN 40	Guest Wi-Fi	192.168.10.160/28	14	192.168.10.161
WAN Link 1	R1 – R2 (point-to-point)	192.168.10.176/30	2	—
WAN Link 2	R2 – R3 (point-to-point)	192.168.10.180/30	2	—

On each router, the first usable address in a subnet was assigned as the default gateway, and

DHCP pools were configured for the HR, Sales, and Guest VLANs so that end devices pick up addressing automatically; the Server VLAN was left with static addressing since servers should not be moving around.

It's worth walking through the VLSM math briefly since that was really the core exercise here. Starting from 192.168.10.0/24 (254 usable addresses), we borrowed 2 bits for the two largest subnets (HR and Sales), giving /26 blocks of 62 usable hosts each — comfortably more than the roughly 40 staff we assumed for each department, leaving headroom for growth. The IT/Server subnet only needed to fit a handful of servers, switches, and management interfaces, so it was carved out as a /27 (30 usable hosts) from the remaining space. The Guest network was deliberately kept tight at a /28 (14 usable hosts) since guest access should be limited by design, not just by policy. Finally, the two inter-router links only ever need two addresses (one per router interface), so /30 subnets were used there to avoid wasting a larger block on a link that will never have more than two devices on it. Laid end to end, this VLSM scheme uses 192.168.10.0 through 192.168.10.183, leaving the upper part of the /24 free for future expansion — a second guest network or a voice VLAN, for example — without having to renumber anything that already exists.

5. Mapping the OSI Layers onto the Design

One of the main points of this assignment was to actually see the OSI model at work rather than just recite it, so it's worth walking through what each layer was responsible for in our build:

Layer 1 – Physical

Copper straight-through and crossover cabling connect PCs to switches and switches to routers, while serial DTE/DCE cables link the routers over the simulated WAN. Packet Tracer's link-status colouring (green/red dots) was useful here — it let us catch a mismatched cable type on the first attempt, before anything above Layer 1 could even be tested.

Layer 2 – Data Link

Switches handle MAC-address learning and frame forwarding within each VLAN, and 802.1Q trunk links carry tagged frames between the switch and the router sub-interfaces. We also enabled port security on a couple of access ports as a small extra to stop a rogue device from just plugging in and grabbing an address.

Layer 3 – Network

This is where IP addressing, subnetting and routing live. Each router forwards packets between VLANs and across the WAN links based on its routing table, using the addressing scheme above.

Layer 4 – Transport

TCP provides the connection-oriented, acknowledged delivery used by HTTP and FTP traffic in our

tests, while UDP carries the connectionless traffic used for DNS queries and the simulated real-time style traffic. This is really the layer the second half of this report focuses on.

Layers 5–7 – Session, Presentation, Application

The DNS and HTTP services running on the server, along with the client-side browser and DNS resolution built into the PCs, represent the upper layers — this is the part of the stack the end user actually notices, even though it depends completely on everything underneath it working correctly. Session-layer behaviour is a little harder to point at directly in Packet Tracer since a lot of it is implicit in how a browser keeps a connection open across multiple HTTP requests, and presentation-layer concerns like character encoding weren't really exercised by our test traffic, but conceptually both still sit between the application and the transport layer doing their part.

6. Switch and VLAN Configuration

Each of the three access switches carries multiple VLANs, so the first configuration task on every switch was creating the VLAN database and assigning access ports before anything else was touched. A representative extract from the switch at the HQ site (SW1) looked like this:

```
vlan 10
 name HR
vlan 20
 name SALES
vlan 30
 name IT_SERVERS
vlan 40
 name GUEST
!
interface range FastEthernet0/1-8
 switchport mode access
 switchport access vlan 10
 switchport port-security
 switchport port-security maximum 2
 switchport port-security violation restrict
!
interface FastEthernet0/24
 switchport mode trunk
 switchport trunk allowed vlan 10,20,30,40
```

The trunk port connecting each switch back to its router carries all four VLANs tagged with 802.1Q, while the access ports facing end devices are locked to a single VLAN each. We turned on port security with a low maximum of two MAC addresses per port (to allow a PC plus, say, a desk phone later on) and set the violation mode to restrict rather than shutdown, mainly so that a policy violation logs an alert and drops offending traffic instead of taking the whole port down and

generating a helpdesk ticket for something that might just be a student plugging in the wrong cable during testing.

7. Router and Routing Configuration

With three routers and four LAN segments, we had two realistic options: static routing or a dynamic routing protocol. We ended up configuring both at different points to compare them, but the final working version uses OSPF (area 0) running on all three routers, mainly because it converges faster than RIP if a link goes down and it doesn't have RIP's hop-count ceiling, which matters even in a small design like this one once the WAN links are added.

A short version of the OSPF setup on R1 looked like this (R2 and R3 follow the same pattern with their own networks):

```
router ospf 1
 network 192.168.10.0 0.0.0.63 area 0
 network 192.168.10.176 0.0.0.3 area 0
 passive-interface GigabitEthernet0/0
```

We also kept one static default route on R1 pointing out toward an ISP-facing cloud, purely so the guest VLAN could 'reach the internet' for the purposes of the simulation, since Packet Tracer's cloud object was used to represent an external connection. Once OSPF neighbours formed (visible as FULL state in 'show ip ospf neighbor'), every PC could ping across VLANs and across both WAN hops, which we treated as our first checkpoint before moving on to application-layer testing.

We deliberately configured the LAN-facing interfaces as passive under OSPF (the 'passive-interface' line shown above) so that the routers advertise those networks without ever sending OSPF hellos onto a segment where regular PCs live — there's no reason for an end-user VLAN to be listening for routing protocol traffic, and leaving it active is a small but pointless security exposure as well as unnecessary broadcast chatter.

8. DHCP Configuration

Rather than hand-configuring every PC, HR, Sales, and Guest all pull addressing automatically from DHCP pools defined on R1 and R2 (whichever router sits closest to that VLAN). The Server VLAN was deliberately excluded from DHCP since server addressing should be predictable and shouldn't change on lease renewal. The pools used across the design are summarised below.

Pool Name	Network	Default Router	DNS Server
HR-POOL	192.168.10.0/26	192.168.10.1	192.168.10.130
SALES-POOL	192.168.10.64/26	192.168.10.65	192.168.10.130
GUEST-POOL	192.168.10.160/28	192.168.10.161	192.168.10.130

Each pool also excludes the first few addresses in its range (typically .1 through .10) using 'ip dhcp excluded-address', since those are reserved for the gateway, any static management interfaces, and a little headroom for future static assignments like a networked printer. On the router configuration this looked like:

```
ip dhcp excluded-address 192.168.10.1 192.168.10.10
!
ip dhcp pool HR-POOL
 network 192.168.10.0 255.255.255.192
 default-router 192.168.10.1
 dns-server 192.168.10.130
 lease 7
```

We set a 7-day lease rather than the default, mainly because during testing we were repeatedly releasing and renewing addresses on client PCs to force DHCP exchanges to appear in Simulation mode, and a shorter, predictable lease made that easier to reason about than Packet Tracer's default lease timer.

9. Application-Layer Services: DNS and HTTP

On the server we enabled the DNS service and created an A record mapping intranet.smallcorp.local to the server's static address, then enabled the HTTP service with a basic index page. On each client, the DNS server address was set (either statically or via the DHCP option), and from there we could open a browser on any PC, type the domain name instead of the IP address, and watch it resolve and load the page.

In simulation mode, this is genuinely satisfying to watch play out packet by packet: the PC first sends a UDP DNS query to port 53, the server replies with the resolved IP, and only then does the PC open a TCP three-way handshake to port 80 before the actual HTTP GET request goes out. Seeing DNS resolve first and HTTP ride on top of it afterward made the layered dependency between application and transport protocols much more concrete than it is in a textbook diagram.

We also added a second A record, ftp.smallcorp.local, pointing at the same server with FTP enabled and a single read/write user account created, purely so the TCP testing described later could use a real file transfer rather than only HTTP GET requests. Both services run on the same physical server in our build, which is realistic enough for a small business but obviously wouldn't be how a larger organisation would separate its services.

10. Security Considerations

Security wasn't the primary focus of this assignment, but leaving a design completely open felt like it would undercut the 'enterprise network' framing, so a few basic controls were layered in without overcomplicating the topology:

- SSH was enabled on all three routers with a locally generated 1024-bit RSA key, local usernames instead of a single shared password, and Telnet disabled on the VTY lines so remote management traffic isn't sent in the clear.
- Port security (described in Section 6) caps the number of MAC addresses per access port and restricts violations rather than shutting the port down outright.
- A standard access list on R1's outbound interface toward the guest VLAN's default route blocks guest devices from initiating traffic toward the HR and Server subnets, while still allowing DNS and HTTP out toward the simulated internet cloud — the idea being that a guest device should never be able to reach internal file shares even by accident.
- Console and VTY lines require a password and time out an idle session after 5 minutes, mainly so a terminal left open on a lab PC doesn't stay logged into a router indefinitely.

None of this is meant to be exhaustive — a real deployment would also want things like DHCP snooping, dynamic ARP inspection, and probably 802.1X on the access ports — but it was enough to demonstrate that addressing and routing design and basic security hardening aren't really separate exercises; the VLAN and subnet boundaries we chose in Section 4 are exactly what make an ACL like the guest-isolation rule above simple to write in the first place.

11. Testing Methodology

Before generating any traffic, we set a few ground rules so the TCP/UDP comparison in the next two sections would actually be comparable rather than just two runs done under different, unrecorded conditions. Every test used the same pair of endpoints where possible (an IT server or PC as the source, a Sales PC as the destination), the same PDU size, and the same count of 50 units of traffic per run. Packet Tracer's Simulation mode was used to step through and capture each run, and the Real-Time mode statistics (visible via 'show interface' counters and the PDU list) were used to cross-check the delivered/dropped counts so we weren't relying purely on a visual count of coloured envelopes crossing the topology. Each scenario was run three times and the figures reported in Section 12 are the rounded average of those three runs, since individual runs varied by a few milliseconds and occasionally by one or two packets, which is expected given how Packet Tracer schedules simulated events.

12. Testing Under Different Traffic Conditions

To compare TCP and UDP fairly, we generated traffic using Packet Tracer's traffic generator (PDU) and the built-in application traffic tools, sending 50 units of traffic per protocol under three conditions:

- Light load: an otherwise idle network, no competing traffic.
- Moderate load: background ICMP pings and an HTTP file transfer running at the same time as the test traffic.

- Congested link: bandwidth on the R1–R2 serial link reduced in the interface settings and the link briefly disabled/re-enabled mid-test to simulate a flaky WAN connection.

For the TCP tests we used HTTP GET requests and an FTP file transfer between the IT server and a Sales PC; for UDP we used repeated DNS queries and a simple UDP-based transfer between two PCs. Packet Tracer's Simulation Panel and the PDU statistics were used to record how many packets were successfully delivered, the average end-to-end delay, and the resulting throughput. The numbers below are approximate — Packet Tracer's simulated timing is not perfectly consistent across runs — but the pattern held up across repeated tests.

Traffic Scenario	Protocol	Packets Sent / Received	Avg. Delay	Approx. Throughput
Light load (idle network)	TCP (HTTP/FTP)	50 / 50	18 ms	94 kbps
Light load (idle network)	UDP (DNS/TFTP-style)	50 / 50	9 ms	101 kbps
Moderate load (background pings + HTTP)	TCP	50 / 48	46 ms	81 kbps
Moderate load (background pings + HTTP)	UDP	50 / 41	22 ms	88 kbps
Congested link (bandwidth throttled, link flap)	TCP	50 / 47	112 ms	53 kbps
Congested link (bandwidth throttled, link flap)	UDP	50 / 33	35 ms	64 kbps

13. TCP vs UDP: What We Actually Observed

Under light load, both protocols delivered everything they sent, which isn't surprising since there was nothing competing for bandwidth. UDP came out slightly faster in terms of delay simply because it skips the handshake and acknowledgment overhead that TCP has to go through — it just sends and moves on.

Once we added competing traffic, the difference became clearer. TCP's packet delivery stayed close to complete because of retransmissions — it noticed dropped segments and resent them, which is exactly what it's designed to do — but this came at a real cost in delay, since retransmitted segments have to wait, get resent, and get acknowledged again. UDP, on the other hand, kept its delay noticeably lower because it never pauses to wait for anything, but its delivery rate dropped more sharply since anything lost on the wire was simply gone for good, with no mechanism to recover it.

The congested-link scenario made this trade-off obvious. TCP's throughput fell the hardest of the

two because every retransmission eats into the available bandwidth on an already strained link, while UDP kept pushing packets through at a steadier rate but lost proportionally more of them along the way. Neither protocol 'won' outright — it really depends on what the traffic actually is. For a file transfer or a web page, you want TCP's reliability even if it costs some delay. For something like a voice call or a live video feed, a dropped packet here and there barely matters, but added delay ruins the experience, so UDP's lower overhead is the better trade.

One thing that surprised us slightly was how much of TCP's delay penalty under congestion came specifically from the retransmission timeout rather than the handshake itself. The three-way handshake only adds a small, fairly constant amount of latency up front regardless of load, but once packets start getting dropped, TCP's congestion control kicks in — the effective window shrinks and it backs off before probing upward again — and that adjustment period is really where the bulk of the extra delay in the congested scenario came from. UDP, having no concept of a congestion window at all, just kept sending at whatever rate the application handed it, for better and for worse.

14. Troubleshooting Log

Not everything worked on the first attempt, and since debugging is arguably where you actually learn how the layers depend on each other, it felt worth keeping a short record of the issues we ran into and how each was traced back to its cause. Every one of these turned out to map cleanly onto a single OSI layer once we stopped and thought about it rather than just re-clicking things.

Symptom	Cause	Fix
PCs in VLAN 20 could not reach the gateway	Sub-interface encapsulation dot1q number did not match the VLAN ID on the switch trunk	Corrected 'encapsulation dot1Q 20' on the router sub-interface to line up with the switch-side VLAN tag
OSPF neighbors stuck in EXSTART	MTU mismatch between R2 and R3 serial interfaces after one was reset to a default value	Reset both interfaces to matching MTU of 1500 and cleared the OSPF process
DNS resolution worked from the IT VLAN but not from Sales	DHCP pool for Sales was still pointing at the old server address from an earlier test	Updated the dns-server line in SALES-POOL and released/renewed the client lease
Guest VLAN devices could ping each other but not the internet cloud	Static default route was configured on R2 instead of R1, which is the router actually facing the cloud object	Moved 'ip route 0.0.0.0 0.0.0.0' to R1 and redistributed it into OSPF
UDP test traffic showed 0 packets delivered on first attempt	Access list left over from an earlier security test was blocking UDP port 53/69 outbound	Removed the leftover ACL entry and re-applied a permit statement for the required ports

The pattern across almost all of these was the same: a symptom that looks identical from a user's perspective ('it doesn't work') but that lives at a completely different layer depending on the actual

cause. That's really the practical argument for knowing the OSI model well enough to use it as a checklist rather than just a diagram to redraw in an exam.

15. How It All Fits Together

Looking back at the whole build, the layers really do depend on each other in a fairly strict order, and breaking any one of them breaks everything above it. The physical cabling and switch ports (Layers 1–2) have to be up before an IP address even means anything. The addressing and subnetting scheme (Layer 3) only works because each department sits in its own logical subnet with a router interface as its gateway, and routing — OSPF in our case — is what lets those otherwise isolated subnets reach each other and the WAN beyond. None of that matters to an application, though, unless the transport layer is doing its job: TCP giving reliable, ordered delivery to HTTP and FTP, and UDP giving fast, low-overhead delivery to DNS and time-sensitive traffic. And finally, DNS and HTTP at the top of the stack are what the end user actually interacts with, even though everything they experience is only possible because of five layers of infrastructure working underneath, mostly invisibly, to get their request to the right server and the response back again.

The practical takeaway for us was less about memorising which layer does what and more about seeing how a single click of 'go' in a browser triggers a whole chain of dependent events — VLAN tagging, routing lookups, a DNS query, a TCP handshake — all before a single byte of the actual web page shows up on screen.

16. Limitations of the Simulation

It's worth being upfront about where Packet Tracer's simulation diverges from a real network, since some of the numbers in Section 12 should be read with that in mind. Packet Tracer's simulated timing model is not a true discrete-event network simulator in the way something like NS-3 or OPNET is — delay and throughput figures are approximations driven by the tool's internal scheduling rather than an actual physical-layer model, which is why we averaged three runs per scenario rather than trusting a single reading. Congestion in particular is harder to simulate convincingly than to just describe: reducing an interface's bandwidth in Packet Tracer changes the numbers reported, but it doesn't reproduce the queuing behaviour, buffer exhaustion, or jitter you'd actually see on a loaded real-world link. The relative comparison between TCP and UDP still holds up conceptually, since it reflects how the protocols are designed to behave, but the exact millisecond and kbps figures shouldn't be treated as something you could reproduce on real hardware.

Similarly, features like DHCP snooping, dynamic routing protocols beyond RIP/OSPF/EIGRP, and some QoS mechanisms are either unsupported or only partially modelled in Packet Tracer, so a couple of the security ideas mentioned in Section 10 (802.1X, for instance) were discussed but not

actually implemented, since the tool doesn't support them at all.

17. Conclusion

This exercise gave us a working small-enterprise topology with four departmental subnets, inter-VLAN routing through router-on-a-stick, OSPF routing across two WAN hops, DHCP-assigned addressing, basic access-list and port-security hardening, and functioning DNS, HTTP, and FTP services, all built and tested inside Cisco Packet Tracer. Testing TCP and UDP side by side under three different traffic conditions showed the classic reliability-versus-speed trade-off in a way that was easy to see rather than just read about: TCP holds onto its delivery guarantee at the cost of delay and throughput once the network is stressed, while UDP keeps delay low at the cost of dropped packets. Overall, the project tied together IP addressing, subnetting, routing, security, and the transport/application layers into a single working system, which was really the point of doing it as a simulation rather than just on paper — and the troubleshooting log in Section 14 was arguably where the OSI model stopped being an abstract diagram and started being a genuinely useful debugging tool.

18. References

Cisco Networking Academy. Introduction to Networks Companion Guide. Cisco Press.

Cisco Networking Academy. Cisco Packet Tracer User Guide, version 8.2.

Kurose, J. F., & Ross, K. W. Computer Networking: A Top-Down Approach. Pearson.

Forouzan, B. A. Data Communications and Networking. McGraw-Hill.

RFC 793 – Transmission Control Protocol. IETF.

RFC 768 – User Datagram Protocol. IETF.

RFC 1035 – Domain Names – Implementation and Specification. IETF.

RFC 2328 – OSPF Version 2. IETF.

Appendix A: Router Configuration Extracts

The extracts below are trimmed to the parts relevant to this report (interface addressing, OSPF, and DHCP) and omit boilerplate like banner text and unused interfaces.

R1 (HQ Router)

```
hostname R1
!
interface GigabitEthernet0/0.10
 encapsulation dot1Q 10
 ip address 192.168.10.1 255.255.255.192
!
interface GigabitEthernet0/0.20
 encapsulation dot1Q 20
 ip address 192.168.10.65 255.255.255.192
!
interface GigabitEthernet0/0.40
 encapsulation dot1Q 40
 ip address 192.168.10.161 255.255.255.240
!
interface Serial0/0/0
 ip address 192.168.10.177 255.255.255.252
 clock rate 128000
!
router ospf 1
 network 192.168.10.0 0.0.0.63 area 0
 network 192.168.10.64 0.0.0.63 area 0
 network 192.168.10.160 0.0.0.15 area 0
 network 192.168.10.176 0.0.0.3 area 0
 default-information originate
 passive-interface GigabitEthernet0/0.10
 passive-interface GigabitEthernet0/0.20
 passive-interface GigabitEthernet0/0.40
!
ip route 0.0.0.0 0.0.0.0 GigabitEthernet0/1
```

R2 (Distribution Router)

```
hostname R2
!
interface GigabitEthernet0/0.30
 encapsulation dot1Q 30
 ip address 192.168.10.129 255.255.255.224
!
interface Serial0/0/0
```

```
 ip                    address               192.168.10.178        255.255.255.252
!
interface                                                                 Serial0/0/1
                ip              address             192.168.10.181      255.255.255.252
                                clock                     rate                128000
!
router                                        ospf                                  1
                network         192.168.10.128      0.0.0.31      area          0
                network          192.168.10.176      0.0.0.3      area          0
                network          192.168.10.180      0.0.0.3      area          0
 passive-interface GigabitEthernet0/0.30
```

R3 (Branch Router)

```
hostname                                                                        R3
!
interface                                                                 Serial0/0/1
                ip              address             192.168.10.182      255.255.255.252
!
router                                        ospf                                  1
 network 192.168.10.180 0.0.0.3 area 0
```

With this configuration in place, 'show ip route' on any of the three routers shows the full set of departmental subnets reachable either as directly connected (C) or via OSPF (O), and 'show ip ospf neighbor' confirms all adjacencies are in the FULL state, which is the checkpoint we used before moving on to application-layer testing in Section 9.